

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The ArtisanHub Handmade Products E-Commerce Database System is designed to efficiently manage customers, artisans, products, categories, orders, payments, deliveries, reviews, wishlists, inventory, and coupons. Functional dependencies help identify the relationship between attributes in each table. Redundancy analysis ensures that duplicate data is minimized, while data anomaly analysis helps maintain data consistency, integrity, and efficient database operations.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address, Password

Redundancy Analysis

Customer information is stored only once in the Customer table. Other tables use Customer_ID as a foreign key instead of repeating customer details.

Data Anomalies

Insert Anomaly: A customer can register without placing an order.

Update Anomaly: Updating a customer's phone number is done only in the Customer table.

Delete Anomaly: Deleting an order does not remove customer information.

2. ARTISAN TABLE

Functional Dependency

Artisan_ID → Artisan_Name, Email, Phone_Number, Shop_Name, Address

Redundancy Analysis

Each artisan's information is stored once. Products reference the Artisan_ID instead of repeating artisan details.

Data Anomalies

Insert Anomaly: A new artisan can register without adding products.

Update Anomaly: Changing the artisan's shop name is updated only once.

Delete Anomaly: Removing a product does not delete artisan information.

3. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name, Description

Redundancy Analysis

Categories are stored only once. Products refer to Category_ID instead of storing category names repeatedly.

Data Anomalies

Insert Anomaly: Categories can be created before adding products.

Update Anomaly: Category names are updated only once.

Delete Anomaly: Deleting a product does not remove the category.

4. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Price, Stock, Description, Category_ID, Artisan_ID

Redundancy Analysis

Product information is maintained only in the Product table. Orders reference Product_ID instead of duplicating product details.

Data Anomalies

Insert Anomaly: Products can be added without receiving orders.

Update Anomaly: Product price or stock is updated only once.

Delete Anomaly: Deleting an order does not remove product information.

5. INVENTORY TABLE

Functional Dependency

Inventory_ID → Product_ID, Available_Stock, Last_Updated

Redundancy Analysis

Inventory details are maintained separately, preventing duplicate stock information in the Product table.

Data Anomalies

Insert Anomaly: Inventory records can be created after adding products.

Update Anomaly: Stock updates affect only the Inventory table.

Delete Anomaly: Deleting inventory records does not remove product information.

6. SHOPPING CART TABLE

Functional Dependency

Cart_ID → Customer_ID, Product_ID, Quantity

Redundancy Analysis

Shopping cart items are stored separately, avoiding unnecessary duplication in the Orders table.

Data Anomalies

Insert Anomaly: Customers can add products to the cart before placing an order.

Update Anomaly: Quantity changes affect only the selected cart item.

Delete Anomaly: Removing a cart item does not delete the customer or product.

7. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order-related information. Customer details are referenced through Customer_ID.

Data Anomalies

Insert Anomaly: Orders can be created after selecting products.

Update Anomaly: Order status is updated only once.

Delete Anomaly: Deleting an order does not remove customer or product records.

8. ORDER_DETAILS TABLE

Functional Dependency

Order_Detail_ID → Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

This table stores product details for each order, eliminating repeated product information in the Orders table.

Data Anomalies

Insert Anomaly: Products can be added to an order individually.

Update Anomaly: Quantity updates affect only one order item.

Delete Anomaly: Removing an order item does not delete the product.

9. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date

Redundancy Analysis

Payment details are stored separately from Orders, avoiding duplicate payment records.

Data Anomalies

Insert Anomaly: Payments can be recorded after an order is placed.

Update Anomaly: Payment status is updated only once.

Delete Anomaly: Deleting payment information does not remove the order.

10. DELIVERY TABLE

Functional Dependency

Delivery_ID → Order_ID, Delivery_Address, Delivery_Status, Expected_Date

Redundancy Analysis

Delivery information is maintained in a separate table to prevent repeating shipping details.

Data Anomalies

Insert Anomaly: Delivery records can be created after payment confirmation.

Update Anomaly: Delivery status is updated independently.

Delete Anomaly: Deleting delivery records does not affect order details.

11. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text

Redundancy Analysis

Customer reviews are stored only once and linked using Customer_ID and Product_ID.

Data Anomalies

Insert Anomaly: Reviews can be added after purchasing products.

Update Anomaly: Review content is updated only once.

Delete Anomaly: Deleting a review does not remove product information.

12. WISHLIST TABLE

Functional Dependency

Wishlist_ID → Customer_ID, Product_ID

Redundancy Analysis

Wishlist entries prevent storing favorite products repeatedly in the Customer table.

Data Anomalies

Insert Anomaly: Products can be added to the wishlist without ordering.

Update Anomaly: Wishlist entries can be modified independently.

Delete Anomaly: Removing wishlist items does not affect products.

13. COUPON TABLE

Functional Dependency

Coupon_ID → Coupon_Code, Discount_Percentage, Expiry_Date

Redundancy Analysis

Coupon details are stored once and reused for multiple customers.

Data Anomalies

Insert Anomaly: Coupons can be created before use.

Update Anomaly: Coupon details are updated only once.

Delete Anomaly: Deleting a coupon does not affect previous orders.

14. COUPON_USAGE TABLE

Functional Dependency

Usage_ID → Coupon_ID, Customer_ID, Order_ID

Redundancy Analysis

Coupon usage records are stored separately, preventing repeated coupon information in the Orders table.

Data Anomalies

Insert Anomaly: Coupon usage is recorded only when applied.

Update Anomaly: Coupon usage details are updated independently.

Delete Anomaly: Removing coupon usage does not delete the coupon or order.

CONCLUSION

The ArtisanHub Handmade Products E-Commerce Database System is designed using proper functional dependencies and normalized database tables. Each entity stores only its relevant information, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains data integrity, and supports efficient management of customers, artisans, products, orders, payments, deliveries, reviews, wishlists, inventory, and coupons.